

TRAN HUY VIET

✉ teiwiet33@gmail.com ☎ +84975972419 🔗 teiwiet.github.io 🌐 github.com/teiwiet

PROFILE

Entry-level security researcher focused on firmware and embedded systems. Self-taught through projects in firmware reverse engineering, vulnerability analysis, and exploit development on MIPS/ARM targets, with a working knowledge of C/C++, Rust, and Linux internals. Motivated to build offensive tooling and contribute through responsible disclosure.

EDUCATION

Bachelor of Engineering in Telecommunications Engineering |

09/2020 – 11/2025

Classification : Good

Posts and Telecommunications Institute of Technology (PTIT)

SKILLS

Languages

C, C++, Rust, Assembly

Reverse Engineering & Analysis

Ghidra, Binwalk, GDB, QEMU, ELF analysis

Security

Firmware reverse engineering, exploit development (MIPS/ARM), OWASP Top 10

Embedded & OS Internals

STM32, Raspberry Pi, Linux Kernel, Bare-metal programming, eBPF/libbpf

PROJECTS

Firmware Exploitation Research 🔗

- Extracted firmware at the hardware level: after PCB UART probing returned only garbled output, dumped firmware directly from SPI flash with a CH341A programmer rather than relying on vendor-supplied binaries.
- Unpacked filesystems with binwalk and emulated services in QEMU (user- and system-mode) for dynamic analysis; recovered hardcoded credentials and private keys through static analysis.
- Identified an unauthenticated authentication bypass and a stack-based buffer overflow in a Tenda AC10 V4.0 router; both reported to VulDB (CVEs pending).
- Published technical writeups : teiwiet.github.io | Advisory : <https://github.com/teiwiet/tenda-ac10-vulnerabilities> 🔗

kprowl — eBPF Dynamic Analysis Sandbox 🔗

- Developed an eBPF program in C to monitor Linux processes at the kernel level, hooking `execve`, `file opens` (`security_file_open`), and `network connects` (`security_socket_connect`), and sending events to a userspace program through a BPF ring buffer.
- Added two modes: one that traces a single program the user launches (and its child processes), and one that watches the whole system for access to sensitive files and outbound connections — switchable through a config map without recompiling.
- Attached BPF programs to kernel tracepoints (`execve`, `openat`, `connect`, `fork/exit`), streaming events to user space through a BPF ring buffer with CO-RE for cross-kernel portability.

Firmware Toolkit 🔗

- Built a CLI toolkit in Rust to automate repetitive steps in IoT/router firmware testing (unpacking, secret scanning, binary analysis, fuzzing), wrapping the workflow into two commands `analyze` and `fuzz`.
- Wrote a static analysis module using `goblin` to parse ELF symbols and flag binaries that use risky C functions (`system`, `strcpy`, `sprintf`) and input functions (`read`, `recv`, `scanf`), then rank them by a simple attack-surface score; used `rayon` to scan files in parallel.
- Added a secret scanner that walks the extracted filesystem to detect sensitive files (`shadow`, `.pem`, `dropbear`) and uses `regex` to find private keys and hardcoded passwords in file contents.
- Implemented a basic mutation-based fuzzer with a process harness that feeds mutated inputs via `stdin` and detects crashes through exit codes (`SIGSEGV/SIGABRT`), saving crashing inputs for later analysis.

Operating System Development (Ongoing) 🔗

- Developing a hobby x86 operating system in C and Assembly.
- Implemented bootloader interaction and early kernel initialization
- Built simple framebuffer-based graphics modules for basic rendering.

WORK EXPERIENCE

Network Maintenance Engineer (Intern)

04/2025 – 10/2025 | Ha Noi

FPT-Telecom

- Supported troubleshooting of routers & switches, handling customer network issues.
- Performed on-site device configuration, firmware upgrades, and network diagnostics for residential customers.

CERTIFICATES

Cisco Certified Network Associate (CCNA)

TOEIC : 865